Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Processing (01CT1513)	**Aim :Perform FFT and IFFT on discrete time signal.**	
Experiment :- 5	**Date: 25/09/2026**	**Enrolment No: 92400133125**

Git Hub : https://github.com/mankumarbhalodiya/Digital-Signal-and-Image-Processing

Aim : Perform FFT and IFFT on discrete time signal.

Code :

```
import numpy as np
import matplotlib.pyplot as plt
signal = np.array([1, 2, 3, 4, 5, 6, 7, 8])
fft_result = np.fft.fft(signal)
fft_real = np.real(fft_result)
fft_imag = np.imag(fft_result)
magnitude_spectrum = np.abs(fft_result)
phase_spectrum = np.angle(fft_result)
reconstructed_signal = np.real(np.fft.ifft(fft_result))
plt.figure(figsize=(10, 12))
plt.subplot(4, 1, 1)
plt.stem(signal)
plt.title("Original Signal")
plt.xlabel("Sample")
plt.ylabel("Amplitude")
for i, value in enumerate(signal):
```

```
    plt.annotate(
        f"{value}",
        (i, value),
        xytext=(0, 8),
        textcoords="offset points",
        ha="center"
    )
plt.subplot(4, 1, 2)
plt.stem(
    fft_real,
    linefmt='C0-',
    markerfmt='C0o',
    basefmt=' ',
    label="Real Part"
)
plt.stem(
    fft_imag,
    linefmt='C1-',
    markerfmt='C1s',
    basefmt=' ',
```

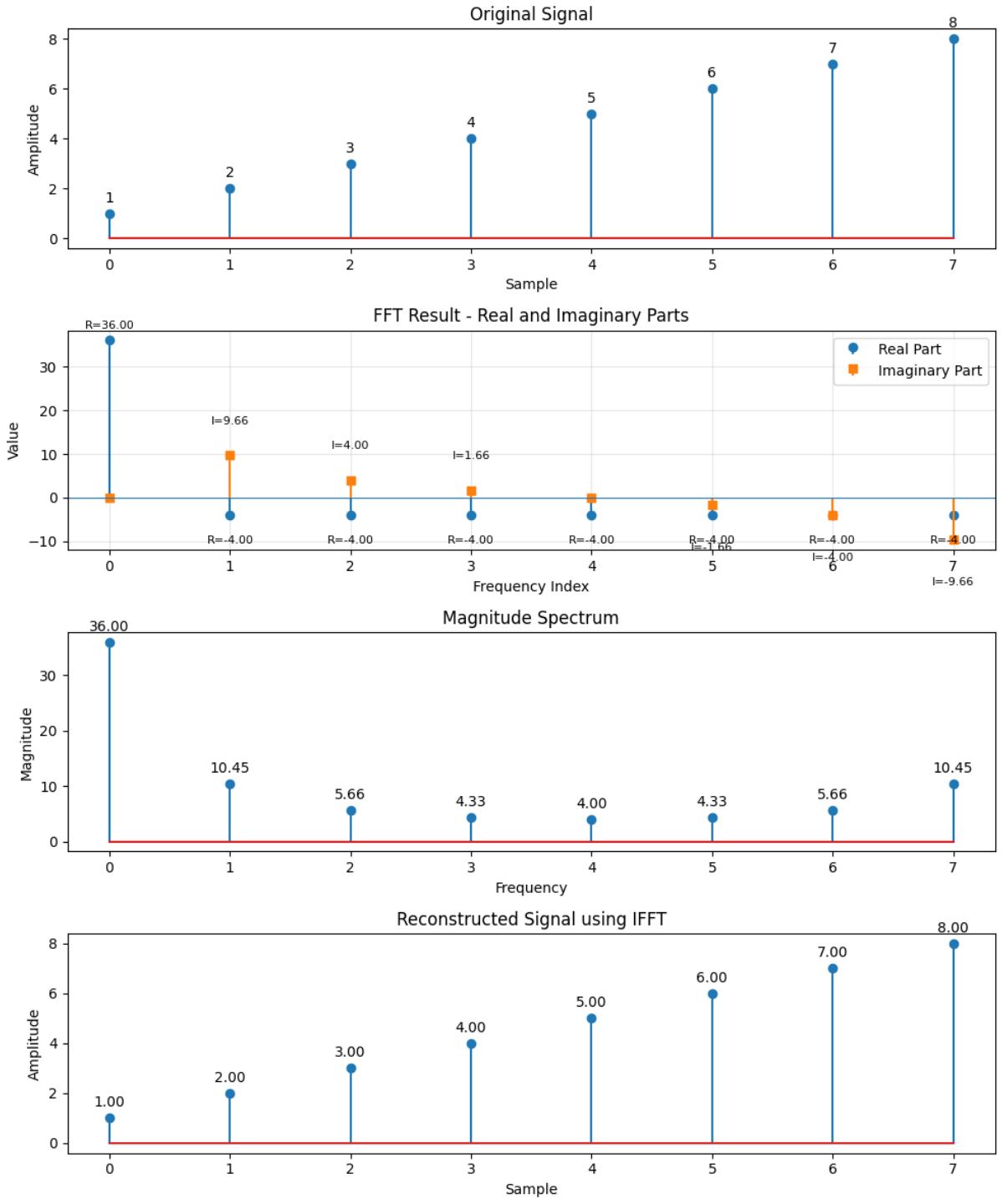
```
    label="Imaginary Part"
)
for i, value in enumerate(fft_real):
    offset = 8 if value >= 0 else -20
    plt.annotate(
        f"R={value:.2f}",
        (i, value),
        xytext=(0, offset),
        textcoords="offset points",
        ha="center",
        fontsize=8
    )
for i, value in enumerate(fft_imag):
    if abs(value) < 1e-10:
        continue
    offset = 22 if value >= 0 else -32
    plt.annotate(
        f"I={value:.2f}",
        (i, value),
        xytext=(0, offset),
```

```
        textcoords="offset points",
        ha="center",
        fontsize=8
    )
plt.axhline(0, linewidth=0.8)
plt.title("FFT Result - Real and Imaginary Parts")
plt.xlabel("Frequency Index")
plt.ylabel("Value")
plt.xticks(range(len(signal)))
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(4, 1, 3)
plt.stem(magnitude_spectrum)
plt.title("Magnitude Spectrum")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
for i, value in enumerate(magnitude_spectrum):
    plt.annotate(
        f"{value:.2f}",
        (i, value),
```

```
        xytext=(0, 8),
        textcoords="offset points",
        ha="center"
    )
plt.subplot(4, 1, 4)
plt.stem(reconstructed_signal)
plt.title("Reconstructed Signal using IFFT")
plt.xlabel("Sample")
plt.ylabel("Amplitude")
for i, value in enumerate(reconstructed_signal):
    plt.annotate(
        f"{value:.2f}",
        (i, value),
        xytext=(0, 8),
        textcoords="offset points",
        ha="center"
    )
plt.tight_layout()
plt.show()
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital Signal and Processing (01CT1513)	**Aim :Perform FFT and IFFT on discrete time signal.**	
Experiment :- 5	**Date: 25/09/2026**	**Enrolment No: 92400133125**

Output :



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Processing (01CT1513)	**Aim :Perform FFT and IFFT on discrete time signal.**	
Experiment :- 5	**Date: 25/09/2026**	**Enrolment No: 92400133125**

Conclusion :

The FFT of the given discrete signal was successfully computed using Python. The magnitude spectrum and phase spectrum were obtained and plotted. The original signal was also successfully reconstructed using the Inverse FFT (IFFT). This experiment demonstrates how FFT can efficiently analyze a discrete signal in the frequency domain and how IFFT can convert it back to the time domain.